Building a Reproducible Platform for Measuring Vegetation from Sentinel-2 Satellite Data

Gordon McWilliams — August 2026

Live platform: ca-oeop-dev-web.politeriver-f001c624.eastus2.azurecontainerapps.io

Executive Summary

The Orbital Earth Observation Platform (OEOP) is a cloud-hosted scientific application that uses Copernicus Sentinel-2 observations to measure vegetation conditions across user-selected areas. The system discovers satellite scenes through a SpatioTemporal Asset Catalog, mosaics and masks the source imagery, calculates the Normalized Difference Vegetation Index, produces temporal and spatial outputs, and records the information required to trace each result back to its source data and processing method.

I built OEOP to explore how my existing experience in Azure, distributed workloads, infrastructure as code, containers, CI/CD, and production operations could be applied to scientific computing and space-derived data.

The project demonstrates that building an Earth-observation application requires more than displaying satellite imagery. A useful scientific system must account for clouds, incompatible raster grids, missing data, sensor characteristics, repeatable scene selection, processing failures, and the provenance of every generated result.

It also taught me something I did not expect to learn so directly: **the most dangerous defects in scientific software are the ones that still produce a plausible-looking answer.** Three of the four significant bugs I found in this project produced charts that looked entirely reasonable. One of them shipped to production and I only caught it because two preview images were different sizes. That story is in the Case Study: A Defect That Looked Fine section, and it is the part of this project I would most want to talk about in an interview.

The Scientific Question

This project asks a straightforward scientific question: **how has vegetation health changed across a selected area over time?**

Vegetation change is useful to measure because it can provide information about seasonal growth, drought stress, agricultural conditions, wildfire recovery, deforestation, land-use change, and broader ecosystem behavior. Satellite observations make it possible to examine these patterns repeatedly across large areas without requiring a physical sensor at every location.

OEOP uses the Normalized Difference Vegetation Index (NDVI) as its primary measurement, calculated from red and near-infrared reflectance:

```
NDVI = (NIR − Red) / (NIR + Red)
```

For Sentinel-2, the platform uses Band 4 for red reflectance and Band 8 for near-infrared reflectance. Both are available at approximately 10 metres per pixel. Sentinel-2's multispectral instrument collects 13 spectral bands at 10, 20, and 60 metre resolutions depending on the band.

The equation works because healthy green vegetation interacts with red and near-infrared light in different ways. Chlorophyll absorbs much of the visible red light that reaches a leaf, while the internal structure of the leaf reflects a significant amount of near-infrared light. A pixel containing dense green vegetation will therefore tend to have a higher near-infrared value than red value, producing a positive NDVI result.

In plain language, NDVI asks:

> *Is this pixel reflecting light more like active green vegetation than like soil, water, snow, cloud, or a constructed surface?*

NDVI values theoretically range from -1 to 1. Negative values are commonly associated with water, snow, clouds, or other non-vegetated surfaces. Values near zero often represent bare soil, rock, built surfaces, or sparse vegetation. Increasingly positive values generally indicate a stronger vegetation signal.

What NDVI does not tell you

NDVI is a proxy, not a direct biological measurement. It does not prove that a specific plant is healthy, identify a species, determine whether vegetation is native or invasive, measure crop yield directly, or establish the cause of a change.

A decrease could result from drought, harvest, fire, seasonal leaf loss, cloud contamination, snow, land clearing, differences in acquisition conditions, or processing errors. Dense vegetation can also cause NDVI to saturate, meaning additional biomass may produce little change in the index.

Repeated observations are therefore more useful than a single image. A time series helps distinguish a persistent trend from an isolated measurement and places each observation within a broader seasonal pattern. Even then, interpretation should be supported by weather records, land-cover information, field observations, or additional spectral indices before making causal claims.

This caution is not decorative. It is enforced in the product: the interface labels every reported change as an observation rather than a cause, and the platform performs no trend fitting or significance testing.

Data Sources

Copernicus Sentinel-2

The primary data source is Copernicus Sentinel-2, a European Earth-observation mission operated as part of the Copernicus Programme. Sentinel-2 satellites use multispectral optical instruments to observe land, inland water, and coastal areas.

OEOP uses Sentinel-2 Level-2A products, which provide atmospherically corrected surface-reflectance estimates together with supporting information such as the Scene Classification Layer. The assets used by the NDVI workflow are:

Asset	Purpose	Resolution
`B04`	Red reflectance	10 m
`B08`	Near-infrared reflectance	10 m
`SCL`	Scene Classification Layer — identifies invalid or unsuitable pixels	20 m
`visual`	True-colour composite, used for human-readable previews only	10 m

Coverage is global but not unlimited: Sentinel-2 observes land roughly between 56°S and 83°N. It does not cover the poles or open ocean.

Microsoft Planetary Computer

Rather than maintaining a separate copy of the Sentinel-2 archive, OEOP discovers and reads data through the Microsoft Planetary Computer, which indexes environmental datasets through a STAC API and exposes raster assets as Cloud Optimized GeoTIFFs.

A Cloud Optimized GeoTIFF (COG) is organized so software can request only the portion of a raster required for an analysis. This matters because a complete Sentinel-2 granule covers roughly 110 km × 110 km, while a typical area of interest here is around 137 km² — well under two percent of the granule. OEOP calculates the raster window intersecting the selected area and reads only that window over HTTP range requests. Full scenes are never downloaded.

SpatioTemporal Asset Catalog

STAC provides a standardized way to describe and search geospatial assets. A STAC Item represents an individual spatiotemporal record as a GeoJSON feature with a geometry, acquisition time, metadata, links, and associated data assets.

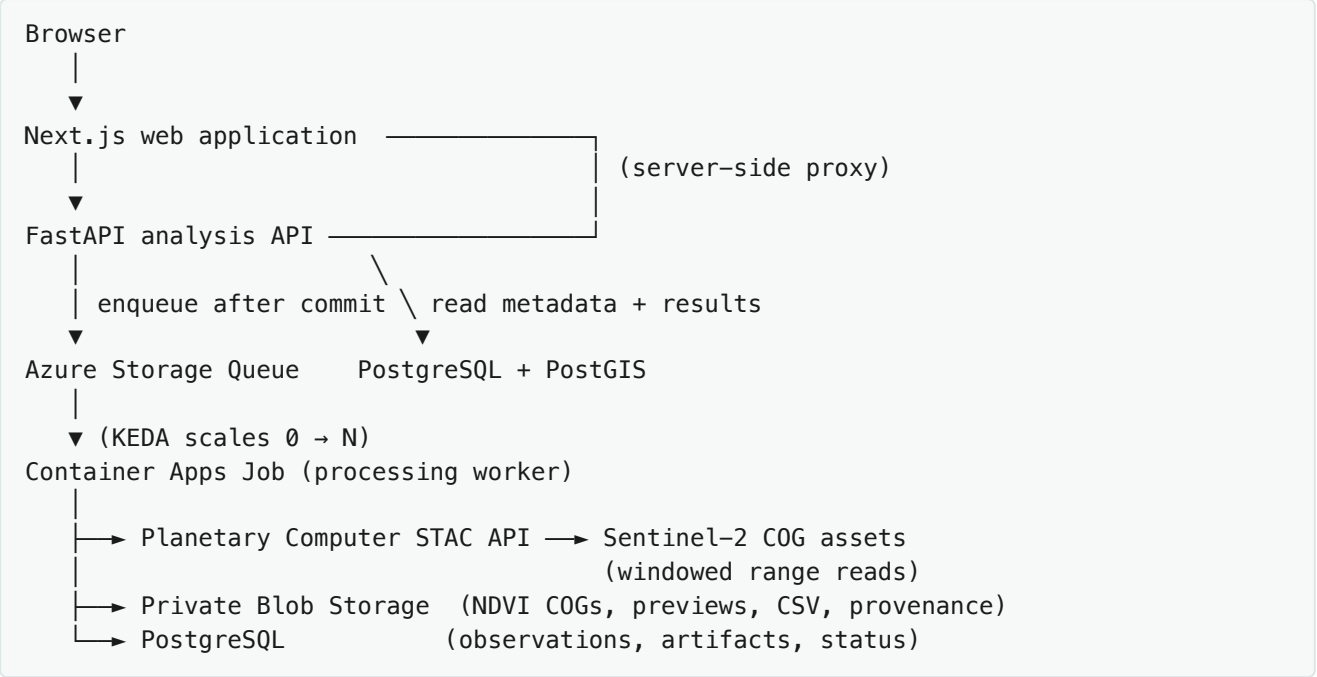
For this project, STAC acts as the discovery layer between the application and the satellite archive. The worker submits a query containing the area of interest, the requested date range, the Sentinel-2 collection, and the maximum acceptable scene-level cloud percentage.

One of the most important lessons from working with STAC was that **metadata filtering is necessary but not sufficient.** A scene may report low overall cloud cover while still containing a cloud directly over the requested area. Scene-level cloud metadata helps reduce the search space, but pixel-level

classification is still required before calculating statistics.

System Architecture

OEOP separates the interactive application from the scientific processing workload.



Web application. The user selects an area, chooses a date range, sets a cloud-cover threshold, and submits the analysis. The interface also presents job status, time-series statistics, preview imagery, output files, and provenance.

Analysis API. Validates the request before any processing begins: that the coordinates form a valid area, the date range is supported, and numeric thresholds are within acceptable limits. It then creates an analysis record and places a message on the processing queue, returning an identifier immediately rather than holding an HTTP request open while satellite data is processed.

Queue. Separates user interaction from longer-running computation. Satellite discovery and raster processing involve external network requests, large arrays, reprojection, masking, statistics, and file creation. Queue-based processing also allows the worker to retry recoverable failures and prevents one long analysis from blocking unrelated requests.

Processing worker. Claims a job, discovers scenes, reads the required raster windows, aligns and mosaics the inputs, masks invalid pixels, calculates NDVI, generates statistics, publishes artifacts, and updates the analysis record. Keeping this in a dedicated worker makes the processing environment explicit and versionable — the same container image runs every job.

PostgreSQL and PostGIS. Stores analysis requests, state transitions, scene records, summary statistics, artifact metadata, and provenance. PostGIS provides geospatial types so areas of interest are stored and validated as spatial objects rather than unstructured coordinate strings.

Blob Storage. Holds generated artifacts: NDVI rasters as COGs, rendered previews, provenance documents, and checksums. Containers are private; downloads are served through short-lived signed URLs generated per request.

Reliability properties

A few decisions in this layer exist specifically to protect scientific results:

- The queue message is sent **only after** the database transaction commits, so a worker can never observe a message without its analysis record.
- Claiming a job is a conditional update, so two workers cannot process the same analysis concurrently.
- Reprocessing is idempotent: a retry deletes prior partial outputs first.
- An analysis is marked succeeded **only after** every artifact and metadata row is durably persisted.
- Errors are classified as user-input, data, transient, timeout, or internal. Transient faults are left on the queue and redelivered after the visibility timeout expires; a message whose delivery count exceeds `max_dequeue_count`, or whose body is malformed, moves to the poison queue. Deterministic failures are recorded terminally with a failure category and the message is deleted, so they are never retried.

Scientific Processing Workflow

1. Derive one canonical analysis grid

Before any imagery is read, the platform derives a single analytical grid from the area of interest: a UTM coordinate reference system chosen from the AOI centroid, 10 m resolution, and bounds snapped outward to the resolution lattice. Every observation in the analysis is reprojected onto this exact grid.

This is the most important architectural decision in the whole system, and I arrived at it by fixing a production bug rather than by foresight. It is explained in the case study below.

2. Search the catalog

The worker queries the Planetary Computer STAC API using the normalized area, requested dates, collection, and cloud threshold.

Long date ranges are searched in consecutive windows of at most 370 days, with the item cap applied per window. A single STAC query returns at most `max_items` results in catalog order, so querying a multi-year range in one call silently covers only part of it — in testing, a request for 2018–2026 over Detroit returned granules from **2022 onward only**. Windowing turned the same request into 345 granules spanning 2018-01-05 to 2026-06-27.

3. Group granules into acquisitions

A single Sentinel-2 acquisition is distributed as **one STAC item per 110 km military-grid tile**. An area that straddles a tile boundary therefore matches several items representing the *same* observation instant.

Items are grouped into acquisitions by observation time (rounded to the minute), platform, relative orbit, and collection. The tile ID and the *processing* timestamp are deliberately excluded from the grouping key — granules of one acquisition are routinely processed at different times, which is precisely why keying on that field would split them incorrectly.

4. Select acquisitions deterministically

Selection follows explicit rules rather than whichever results arrive first. Two strategies are available, and the choice materially changes what the resulting series means:

Temporal spreads observations evenly across the range. Good for watching a single growing season.

Seasonal takes one observation per year from the same part of the calendar. This is the only sound way to compare *across* years, and the reason is worth stating plainly: in a temperate region NDVI swings from roughly 0.15 (dormant) to 0.85 (peak canopy) within one year, while a multi-year trend is on the order of 0.02–0.05. Spreading eight scenes evenly over eight years picked June, February, April, May, April, October, June, May — a "trend" from that series is dominated by *which month each scene happened to fall in*, by roughly an order of magnitude.

Every excluded acquisition is recorded with a reason.

5. Mosaic onto the canonical grid

Every granule intersecting the grid is read over its window only and reprojected onto the canonical grid. Overlaps resolve deterministically: granules are consumed in ascending item ID, and the first one to supply a pixel keeps it. Within a single acquisition the overlapping granules observe the same ground at the same instant — same datatake, same downlink — but each tile is orthorectified onto its own UTM grid, so the candidates agree to within resampling noise rather than exactly. The rule is chosen for determinism, and the tests assert that no seam appears at the join.

Resampling method is chosen per data type, and this distinction is not optional:

Data	Method	Why
Red, NIR (continuous reflectance)	Bilinear	Avoids aliasing when source and target grids are offset. Applied identically to both bands before the ratio, so NDVI is not systematically biased.
Scene Classification Layer (categorical)	**Nearest**	Mandatory. Averaging class labels invents classes that do not exist — interpolating cloud (9) and vegetation (4) would yield water (6).

6. Validate coverage

Geometric AOI coverage is computed per acquisition. Anything below the configured threshold (99% by default) is marked unusable and excluded, **before** any cloud statistics are considered. A partially observed date is not comparable to a fully observed one no matter how clean its pixels are.

Every AOI pixel is then classified exactly once, in a precedence order that follows the physical chain — you cannot have cloud over ground the sensor never saw:

1. **Uncovered** — no source granule reached this pixel

2. **Nodata** — covered, but the source carried no value (SCL class 0, or non-finite SCL or reflectance)
3. **Cloud / shadow / cirrus** — masked by SCL policy
4. **Snow / ice** — masked by SCL policy
5. **Other masked** — saturated or defective
6. **Invalid spectral** — zero NDVI denominator (degenerate reflectance)
7. **Valid** — contributes to statistics

This means a low valid-pixel percentage can always be attributed to a specific cause rather than appearing as an unexplained gap.

7. Convert digital numbers to reflectance

Sentinel-2 L2A distributes reflectance as scaled integers. Since **processing baseline 04.00** (January 2022), the encoding includes an additive offset:

```
reflectance = DN × 1e−4 − 0.1      for baseline ≥ 04.00
reflectance = DN × 1e−4            for earlier baselines
```

This is a genuine trap. NDVI is a ratio, so a common *multiplicative* scale cancels — which makes it tempting to skip scaling entirely. But an *additive* offset does **not** cancel. Ignoring it materially biases the index, and mixing pre- and post-04.00 scenes in one time series without handling it would create a spurious step change at the baseline boundary that has nothing to do with vegetation.

8. Mask, then calculate NDVI

The worker builds a validity mask from the SCL policy, nodata values, non-finite values, and the AOI geometry, then computes NDVI in float64 and stores float32.

The default mask policy is deliberate and documented per class. Cloud, cloud shadow, cirrus, snow, nodata, and saturated/defective pixels are removed. Terrain shadow and **water are retained** — water is a real surface with legitimately negative NDVI, and masking it would misrepresent any area containing lakes or rivers.

Masking is not cosmetic. If clouds are included, their red and near-infrared values shift the average and create a false signal of vegetation change.

9. Compute statistics over the canonical footprint

Statistics are computed over valid pixels inside the **canonical AOI mask** — never over whatever the source granules happened to cover. Reported per observation: mean, median, min, max, standard deviation, the 10th/25th/75th/90th percentiles, valid and masked pixel counts, and valid-pixel percentage.

The mean provides a simple time-series value, but it should not be mistaken for a complete description of the area. Two scenes can have the same average while containing very different spatial patterns.

10. Write outputs and record provenance

Each observation produces a float32 NDVI COG (structurally validated), a colorized NDVI preview, a true-colour preview, and a summary JSON. Each analysis additionally produces a time-series CSV, a summary JSON, and a schema-validated provenance document.

Preview images are useful for communication, but the raster remains the more scientifically valuable product because it preserves pixel values, coordinates, resolution, and nodata information.

Case Study: A Defect That Looked Fine

This is the part of the project I would most want to discuss.

A deployed analysis over central Detroit produced before/after previews with visibly different extents — the earlier image covered only the northern portion of the area, the later one reached the Detroit River. It looked like a CSS problem.

It was not.

Root cause. The Detroit area of interest straddles the boundary between Sentinel-2 tiles **T17TLG** and **T17TLH**. Each acquisition publishes one item per tile:

- T17TLG granules covered **100%** of the AOI
- T17TLH granules covered **56.2%** (northern portion only)
- **13 of 24** acquisitions had *both* granules available — the data for full coverage existed and was being discarded

The pipeline selected one STAC item per time bucket and required only 25% AOI overlap, so a 56%-coverage granule was accepted as a complete observation. The raster read then used a clip operation that silently intersects the requested window with the granule's own extent rather than failing.

What it produced. Rasters of 1272×627 pixels for T17TLH dates against 1272×1149 for T17TLG dates, and NDVI statistics computed over **751,081 pixels versus 1,372,771 pixels** — different ground.

Quantifying the error. Reprocessing isolated it cleanly, because full-coverage dates should be unchanged and partial-coverage dates should move:

Acquisition	Old coverage	Old mean NDVI	Corrected	Change
2026-02-27 (T17TLG)	100%	0.1574	**0.1574**	**0.0000** — unchanged, as expected
2026-05-30 (T17TLH)	56.2%	0.4233	**0.3671**	**−0.0562** — inflated

The full-coverage date reproduced *exactly*, confirming the correction was targeted rather than a wholesale change in the science. The partial date moved by −0.056 because the original measured only the northern 56% of the area, which is greener and less densely built than the riverfront it omitted.

Since the series interleaved 56%- and 100%-coverage dates, its date-to-date differences mixed a real vegetation signal with a change of measurement area of comparable magnitude. **The change previously reported for that analysis was not a valid measurement.**

The fix is the canonical analysis grid described earlier, plus acquisition-level mosaicking and the coverage gate. Every usable observation now shares one CRS, transform, size, and AOI mask by construction, and the worker refuses to publish an analysis whose observations disagree.

Why the tests missed it. The synthetic fixtures used a single scene whose extent fully contained the AOI, so the truncation path was never exercised. The suite now includes adjacent-granule fixtures that reproduce a tile-crossing area, plus assertions that no seam appears, that partial-coverage acquisitions are rejected, and that COG geometry, preview dimensions, and pixel counts are identical across dates.

Three more bugs, only one of them loud

Unbounded NDVI. Once the baseline-04.00 offset was removed, some pixels had slightly negative reflectance (a normal retrieval artifact over water and deep shadow). A near-zero denominator then produced NDVI values of arbitrary magnitude — one April scene reported a mean "NDVI" of about 2.5×10^8. Clipping negative reflectance to zero before the ratio guarantees the result stays within $[-1, 1]$.

Silently truncated searches. Described above: an eight-year request that quietly analysed four years.

A leap-year off-by-one. The seasonal window used a hardcoded 365-day year, shifting the target date by one day in leap years. Caught by a test, not by inspection.

None of the four crashed, and three produced output a reasonable person would have accepted. The unbounded-NDVI defect is the exception that proves the point: it was the only one loud enough to announce itself, and it was also the only one I found within minutes. The defences that actually worked on the other three were type checking, adversarial test fixtures, and comparing results against a value I could predict independently.

Reproducibility Model

An attractive satellite map is not automatically a reproducible scientific result. Without information about the source scenes and processing decisions, it may be impossible to determine why a value was produced or to repeat the analysis later.

For each result, OEOP records:

- Source collection and STAC endpoint
- **Every** contributing STAC item ID and Sentinel-2 tile ID
- Acquisition (sensing) times — distinct from processing times
- Original **unsigned** asset references
- Area-of-interest geometry and geodesic area
- Requested date range and cloud threshold
- The canonical grid: CRS, resolution, affine transform, width, height, bounds

- Scene-selection algorithm, version, and every exclusion with its reason
- Included and excluded SCL classes, by ID and name
- Mosaic method and the resampling method for spectral and categorical data
- Reflectance scaling applied per acquisition and where it came from, the processing baseline of every contributing granule, and a warning when those baselines disagree
- Processing version, Git commit SHA, container image, Python version, dependency lockfile hash
- Output filenames, sizes, and SHA-256 checksums
- Job timings and any warnings

Provenance documents are validated against a published JSON Schema before being persisted.

A note on signed URLs: Planetary Computer assets require time-limited signed references. The platform signs immediately before reading and **never persists a signed URL as provenance** — the durable identifiers are the STAC item IDs, collection name, and asset keys. A short-lived URL would not be a dependable long-term scientific identifier.

This model separates three questions that are often conflated:

1. What did the user request?
2. Which observations did the system use?
3. What code and processing rules transformed those observations into outputs?

Reproducibility does not guarantee that an interpretation is correct. It guarantees the result can be inspected, challenged, and compared with another implementation. That is an essential property of trustworthy scientific software.

Results

Example: seasonal vegetation cycle, Southeast Michigan

Parameter	Value
Region	Southeast Michigan Demonstration Region (Oakland County)
Area of interest	−83.30, 42.55 → −83.15, 42.65 (136.75 km²)
Requested date range	2024-04-01 to 2024-10-31
Maximum scene cloud cover	30%
Canonical grid	EPSG:32617, 1261 × 1144 px @ 10 m (1,442,584 grid cells)
AOI pixels per observation	1,367,491 (grid cells inside the AOI polygon)
Candidate acquisitions	39
Usable observations	6
Excluded	33 (22 temporal sampling, 11 insufficient AOI coverage)
Mean valid-pixel percentage	99.78%
Output artifacts	26 files, 50.2 MB

Observation	Mean NDVI	Median	Valid pixels	Granules
2024-04-13	0.444	0.460	99.95%	2
2024-05-31	0.609	0.682	99.95%	2
2024-06-12	0.593	0.667	100.00%	2
2024-07-27	0.600	0.673	100.00%	2
2024-08-24	0.587	0.655	98.80%	2
2024-10-05	0.582	0.656	100.00%	2

Mean NDVI changed from **0.444 to 0.582**, a difference of **+0.137**. The final usable observation contained a stronger average vegetation signal than the first within the valid pixels of the selected area.

Interpretation. This is consistent with a normal temperate deciduous growing season: a rapid rise from April to late May as leaves emerge, a plateau through summer, and the beginning of decline by early October. The analysis demonstrates a difference in satellite-observed greenness between the selected dates. It does not, by itself, identify the cause.

Worth noting: **every one of these six observations is a mosaic of two granules spanning two different UTM zones** (tiles T16TGN and T17TLH). Before the canonical grid existed, this analysis produced outputs in two different coordinate reference systems.

Example: multi-year comparison

A second analysis, over the Detroit Urban Core region, used the seasonal strategy across 2018–2026 to anchor every observation to the same part of the calendar. It returned one early-July observation per year — 2022 left as a gap, because nothing that year was usable — with mean NDVI between 0.346 and 0.393 and a first-to-last difference of **+0.017**.

That difference is **smaller than the 0.047 scatter within the same series**, so it does not establish a direction of change. Reporting it that way, rather than as a trend, is the point. The platform performs no trend fitting or significance testing, and the documentation says so explicitly.

Global reach

The platform ships ten curated regions across five continents and both hemispheres, most around 137 km² (Hartwick Pines Forest is smaller, at 84 km²), each with a real analysis of live imagery. NDVI ranges are physically sensible per biome:

Region	NDVI range observed
Hartwick Pines Forest (Michigan, conifer)	0.46 – 0.81
Sleeping Bear Dunes (Michigan, dune/water)	0.08 – 0.30
Mekong Delta Rice (Vietnam)	0.42 – 0.69
Doñana Wetlands (Spain)	0.19 – 0.61
Amazon Deforestation Frontier (Brazil)	0.32 – 0.59
Okavango Delta (Botswana)	0.23 – 0.44

The Nile Delta region is a useful stress test: it spans **four** Sentinel-2 tiles, all mosaicked onto one grid, with 423 candidate acquisitions narrowed to 6 usable April observations across 2017–2024.

What I Learned

Remote-sensing data is not ordinary application data. Every raster has a coordinate reference system, transform, pixel size, nodata definition, spectral meaning, acquisition condition, and processing history. Even assets from the same collection require careful handling of grid alignment, masking, and scaling.

Metadata filtering is not enough. A scene with little cloud across the full tile can still be unusable over a small area of interest. OEOP uses scene-level metadata for discovery and pixel-level classification for the calculation.

Every pixel represents a measurement. A map preview makes raster processing look like image manipulation, but the arrays represent measurements tied to locations on Earth. The worker is not changing the colour of an image — it is reading red and near-infrared measurements for each location, testing whether that location is valid, applying an equation, and writing a new georeferenced measurement.

Spatial averages hide local change. Vegetation could decrease in one half of an area and increase in the other while producing almost no change in the mean. This directly motivates a per-pixel change map.

Reproducibility must be designed in. Provenance cannot be bolted on later by saving application logs. Scene identifiers, parameters, algorithm versions, checksums, and software versions have to be captured while the analysis runs. Once lost, recreating them from a rendered map may be impossible.

Plausible output is not correct output. This is the lesson I did not expect. A crash is a gift — it tells you exactly where to look. The defects that cost me the most were the ones that produced a chart someone would have believed. The only reliable defences were making the code assert its own invariants, writing fixtures adversarial enough to break it, and checking results against values I could derive independently.

Infrastructure has value when it protects scientific integrity. The queue, worker, database, storage, containers, monitoring, and IaC modules are not valuable because they form a sophisticated architecture. They are valuable because they make the workflow traceable and reliable — preventing duplicate processing, preserving state, retaining outputs, surfacing failures, and making it possible to determine which code produced a result.

This project changed how I think about platform engineering. **The platform is not the final product. It is the system that protects the integrity of the computation.**

What Comes Next

There is a lot you can do with satellite data and some mathematics. I am new to scientific programming, and this is primarily a platform for me to learn in public and show what I can build with code, AI assistance, and cloud infrastructure.

The next priorities are scientific capability rather than architecture:

Per-pixel change maps. The most valuable immediate addition — a spatial product showing `NDVI_later − NDVI_earlier`. This was genuinely *impossible to do correctly* before the canonical grid, because subtracting two rasters that do not share a grid is meaningless. Now that alignment is guaranteed, it is a small piece of work and would show *where* change occurred rather than only whether the area-wide mean moved.

NDWI for water analysis. The Normalized Difference Water Index would demonstrate that the processing architecture supports a scientific question beyond vegetation. The grid, mosaic, coverage, and provenance machinery is index-agnostic — only the band math changes.

NBR for burn analysis. The Normalized Burn Ratio uses near-infrared and shortwave-infrared information to examine burned areas and fire severity. A pre-fire and post-fire comparison would be a meaningful extension of the change-analysis model.

Polygon areas of interest. Supporting user-drawn polygons would let analyses follow rivers, fields, watersheds, burn perimeters, parks, or property lines rather than rectangular bounding boxes.

Open-source contribution. This project introduced me to STAC, Rasterio, geospatial Python, COG tooling, and the wider Earth-observation software ecosystem. Contributing a focused fix, test, or documentation improvement to one of those projects is a natural next step.

Professional Relevance

OEOP combines my existing experience in Azure platforms, containers, Terraform, CI/CD, security, distributed workloads, observability, and production operations with new experience in geospatial processing, multispectral imagery, Earth-observation data, raster mathematics, scientific provenance, and reproducible analysis.

The project demonstrates that I can do more than deploy infrastructure around a workload. I can learn the scientific meaning of the data, translate that meaning into a processing workflow, **identify where software decisions could invalidate a result**, and build the operational controls required to make the computation reliable.

That middle capability is the one I would emphasize. Finding the tile-boundary defect required noticing that two images were different sizes, suspecting it was not a display bug, and then being able to audit the raster dimensions, pixel counts, and coverage percentages to prove it was a measurement error. Fixing it required understanding both the MGRS tiling scheme and the reprojection machinery. Verifying the fix required constructing a test where I knew the answer in advance.

This is the type of work I am pursuing in scientific computing, Earth observation, mission software, geospatial platforms, and space-related platform engineering.

Technical Summary

Science	Python 3.12, Rasterio, rioxarray, NumPy, Shapely, PyProj, pystac-client, rio-cogeo
Backend	FastAPI, SQLAlchemy 2, Alembic, PostgreSQL + PostGIS, Pydantic v2
Frontend	Next.js 15, React 19, TypeScript (strict), MapLibre GL, Recharts, Zod
Infrastructure	Azure Container Apps + Jobs, Storage Queue, Blob Storage, Key Vault, PostgreSQL Flexible Server, ACR, Log Analytics, Application Insights
Operations	Terraform, GitHub Actions, OIDC federation (no client secrets), managed identity, OpenTelemetry, structured JSON logging
Verification	210 Python tests, 163 frontend tests, Playwright end-to-end, mypy, Ruff, Terraform validate, gitleaks, container image checks

Source code: github.com/raveheart1/Orbital-Earth-Observation-Platform